AR-Based Intelligent Automobile Repair Learning
& Evaluation System

COMMON INTEGRATED SOLUTION REPORT

Weerakoon Y.P.B.	**IT22284570**	AR-Supported Task Verification and Instruction System
Jayalath S.P.S.C.K	**IT22231864**	Student Task Recognition Based on Identified Component
Senevirathne S.D	**IT22279934**	Vision-Based Engine Component Identification
Leelananda M.P.D.R.I	**IT21804960**	Practical Evaluation & Safety Feedback

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October, 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Weerakoon Y.P.B.	**IT22284570**		
Jayalath S.P.S.C.K	**IT22231864**		
Senevirathne S.D	**IT22279934**		
Leelananda M.P.D.R.I	**IT21804960**		

Supervisor certification. The above candidates carried out this research project under my supervision.

Supervisor: Ms. Anjalie Gamage		
Co-supervisor:Ms.Uthpala Samarakoon		

ABSTRACT

This report introduces a system that uses augmented reality and artificial intelligence to help people learn and get evaluated on car repairs. The system is meant to fix the problems that come with having a person watch over trainees all the time and the dangers that can happen during training. The system has four parts that work together. One part finds car parts using a camera. Another part sees what the student is doing. A third part gives instructions using augmented reality. The last part checks how well the student does and gives feedback on safety. The system works without needing the internet. It uses AI tools on phones running Android. The system was tested on a Suzuki Alto 800L car. The results show that the system works fast with response times between 78 and 150 milliseconds. It catches safety issues well. It also gives a score that matches what experts would give. The score has a high connection, to what experts think. This shows that the system can be used for training people in an effective way. This system can be used in places.

Keywords: Augmented Reality, Edge AI, Vocational Training, Automated Evaluation, Automobile Maintenance.

ACKNOWLEDGEMENTS

We want to say a thank you to our Supervisor Ms. Anjalie Gamage, our Co-Supervisor, Ms. Uthpala Samarakoon and our External Supervisor, Dr. Dharshana Kasthurirathna for the priceless help steady support and useful feedback that we received during the whole work on this integrated research project. We also thank Dr. Dinuka Wijendra and the respected Research Panel of the Sri Lanka Institute of Information Technology (SLIIT) for the academic reviews and technical suggestions. We are also very grateful, to the instructors, certified technicians and vocational students who gladly took part in our real-world studies and gave us key industry knowledge. Finally we thank SLIIT for giving us the lab supplies and technical tools that allowed the AutoLearnAR system to finish and work together successfully.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1

1.1 Background and context 1
1.2 Stakeholders and user needs **Error! Bookmark not defined.**
1.3 Alignment with the Sustainable Development Goals 4
1.4 Purpose and scope of the common report 5
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 6
2.1 Overall problem definition 6
2.2 Root causes and overall research gap 7
2.3 Subproblem and component mapping 8
2.4 Overall aim and group objectives 10
2.5 Scope assumptions and constraints 12
3 OVERALL SOLUTION STRUCTURE 14
3.1 Integrated solution overview 14
3.2 High level architecture 15
3.3 Component 1 summary 16
3.4 Component 2 summary 16
3.5 Component 3 summary 16
3.6 Component 4 summary 17
3.7 End to end user and data flow 17
4 COMPONENT INTEGRATION AND COHESIVE OPERATION 18
4.1 Integration strategy 18
4.2 Interfaces data contracts and shared models 19
4.3 Shared nonfunctional requirements 21
4.4 Integrated workflow and exception handling 22
4.5 Team contribution and dependency matrix 23
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 24
5.1 Common technology stack and infrastructure 24
5.2 Development and integration process 26
5.3 Deployment and operational configuration 27
5.4 Evidence of team execution 28
6 INTEGRATED TESTING EVALUATION AND VALIDATION 29
6.1 Evaluation framework and success criteria 29
6.2 Integration and end to end testing 31
6.3 Overall results 33
6.4 Stakeholder and user validation 35
6.5 Achievement of common objectives 37
6.6 Integrated limitations and threats 39
7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 41
7.1 Value proposition and beneficiaries 41
7.2 Market alternatives and differentiation 43
7.3 Business model and feasibility 45
7.4 Adoption intellectual property and compliance 46
7.5 Sustainable Development Goal contribution 48
8 CONCLUSION AND FUTURE DIRECTIONS 50
8.1 Overall conclusion 50
8.2 Principal integrated contribution 51
8.3 Future directions 52
REFERENCES 53
APPENDICES 54
Appendix A Integrated interface specification 54
Appendix B Team contribution declaration 54
TEMPLATE CONVENTIONS 55

LIST OF FIGURES

[Guidance - delete before submission: Use Word's Insert Caption command with the label Figure. Number figures by chapter and cite every figure in the text.]

No table of figures entries found.

LIST OF TABLES

[Guidance - delete before submission: Use Word's Insert Caption command with the label Table. Number tables by chapter and cite every table in the text.]

No table of figures entries found.

LIST OF ABBREVIATIONS

[Guidance - delete before submission: List abbreviations alphabetically. Define each abbreviation at first use in the main text even when it appears here.]

[ABC]	[Full term]
[ABC]	[Full term]
[ABC]	[Full term]

1 INTRODUCTION AND PROJECT CONTEXT

- Background and context

Vocational training is a very practical field. Hands-on work with machines and tools is essential.
Traditionally this training depends on human supervision. Certified instructors watch trainees while they perform repair tasks.
This method has serious problems with scaling and safety. Instructors cannot watch every action of each student at once. This leads to performance reviews and a reactive instead of proactive response to workshop hazards.
While Augmented Reality (AR) and Artificial Intelligence (AI) promise real-time guidance and automatic evaluation, current enterprise-grade systems (for example the Microsoft HoloLens) are far too expensive for technical colleges.
Moreover these systems usually need high-bandwidth cloud connections, which are rarely found in normal automotive garage settings.
Thus there is a gap for a decentralized offline-first digital supervision system.
The AutoLearnAR project fills this gap by designing an Edge AI and AR training solution. It works entirely on affordable consumer-grade devices, such as the Redmi 9C and it keeps automotive education safe, objective and scalable.

1.2 Stakeholders and user needs

- Primary Stakeholders (End-Users):

- Vocational Trainees (User Persona: 'The Novice Mechanic'): These students do not have much practical experience. They need step-by-step help. Their main needs are: reduce effort by not having to flip between printed manuals and tools get real-time feedback on their mechanical work and stay safe while handling dangerous automotive parts.
- Certified Instructors (User Persona: 'The Evaluator'): These teachers have to grade students and keep the workshop safe. Their needs are: reduce the load of watching each student closely get objective data-based performance scores for grading and have an automatic safety system that stops student mistakes before injuries happen.

- Secondary Stakeholders:

- Vocational Training Institutes and Technical Colleges: These schools need scalable learning tech. Their main need is a solution that does not require infrastructure changes so they do not need fast workshop Wi-Fi or pricey AR headsets.
- Automotive Industry Employers: These employers are not users of the app but they matter. Their need is confidence that the graduates they hire have been judged objectively and strictly, on safety and procedure rules.

- Alignment with the Sustainable Development Goals

SDG 4: Quality Education (Targets 4.3 and 4.4):
By running AI checks and AR graphics entirely on simple mobile phones (for example a Redmi 9C) AutoLearnAR makes advanced objectively checked training available to all. It removes the cost of AR headsets giving affordable and fair access to technical skills that lead to jobs and new businesses.
SDG 8: Decent Work and Economic Growth (Target 8.8):
Automotive maintenance has physical risks. AutoLearnAR’s Safety Rule Engine watches tool-to-part moves. Shows AR hazard alerts in real time. This proactive help keeps trainees safe directly supporting healthy work environments for everyone.

- Scope of the common report

The purpose of this report is to show the main problem the whole design and the combined group evaluation of AutoLearnAR. It shows how four separate research parts—vision-based component identification, student task recognition, AR-supported task verification and practical evaluation with safety feedback—work together to create a decentralized first digital supervisor.
This report only covers how the whole system behaves the data flow, common non-functional needs and the overall commercial and sustainable benefits of the full solution. If you want methods, designs, tuning or experiments, on each part you should read the four separate theses written by the team members.

- Alignment with the Sustainable Development Goals

The AutoLearnAR project directly aligns with and actively contributes to two United Nations Sustainable Development Goals (SDGs):

- SDG 4: Quality Education (Targets 4.3 and 4.4): By executing heavy AI inference and AR rendering entirely on consumer-grade mobile devices (e.g. Redmi 9C) the system makes advanced objectively evaluated vocational training available to more people. It removes the cost of proprietary AR headsets and the need for constant cloud connectivity. This ensures that technical and vocational skills needed for jobs are accessible and affordable for everyone.

- SDG 8: Decent Work and Economic Growth (Target 8.8): Automotive maintenance comes with physical risks. The system's deterministic Safety Rule Reasoning Engine checks how tools interact with components. Gives real-time AR warnings, about dangers. This helps protect trainees before any accidents happen. This approach helps create safe and secure working environments for people learning skills.

- Purpose and scope of the common report

The main goal of this report is to describe the big problem we are trying to solve how our solution is put together and what we learned from testing the AutoLearnAR system as a whole. It shows how four separate parts—identifying components using vision recognizing what students are doing verifying tasks with augmented reality and testing everything with safety feedback—work together smoothly. The result is a system that runs completely on its own without needing internet access acting as a supervisor for learners.

This report focuses on how the entire system behaves as one unit—the full path data takes, shared performance goals and the overall impact on the market and sustainability. It does not go into the details behind any single part. For example you won't find explanations, about algorithms, tuning of settings or individual test results. Those details are left out because they belong in documents. If you want to explore the workings of each piece the four team members have written their own theses covering these topics in depth.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

The main issue in vocational training is that automotive vocational training depends mostly on constant one-to-one human supervision to make sure teaching is correct students stay safe and progress is measured.

This old apprenticeship style cannot grow easily because automotive vocational training schools are enrolling students while still having too few qualified instructors.

As a result when many people work in a workshop human supervision becomes subjective and tiring which means grades and assessments differ from one trainee, to another.

Importantly watching students manually is a reactive process, not a proactive one. An instructor who is watching students at once cannot always act fast enough to stop a new trainee from doing something dangerous.

That lag puts both the trainees and the pricey automotive machines at risk of accidents and damage.

In short the real problem is that automotive vocational training does not have a objective and proactive safety and supervision system.

2.2 Root causes and overall research gap

The main reasons why supervision and safety problems happen in automotive training are the limits of human watching and the gap in thinking that shows up with old teaching tools. Trainees normally use printed manuals or digital files and they must keep looking away from the real 3-D engine bay. This makes their brains work harder. Raises the chance of mistakes. At the time instructors who try to watch many students get tired from watching and cannot react fast enough to stop sudden dangerous moves.

There are digital tools today but they cannot fix the real problems. Video lessons are easy to use. They cannot check if the trainee is doing the right physical steps or keep the workshop safe at the moment. Big company tools like the Microsoft HoloLens give AR help but they cost a lot and need strong internet links that small schools and garages do not have.

So the big problem we need to solve is that there is no offline digital supervisor that can guide and check trainees on everyday phones. Fixing this needs than one idea; we need a four-part system. A full digital supervisor must first be able to look and recognize the work area (Component 1). Then it must understand where the trainee is, in the steps (Component 2). Next it must point out where the trainee should go and confirm that the trainee is doing the moves (Component 3). Finally it must stop actions before they happen and give a clear score of performance (Component 4).

2.3 Subproblem and component mapping

To effectively replace ways of people watching and guiding trainees with a digital system the big problem needs to be broken down into four separate parts that don’t overlap but still work closely together. Each part solves a problem that happens when people learn how to fix cars. Each part connects directly to one of the four parts of the AutoLearnAR system.

- Subproblem 1: Visual Context Acquisition (Connected to Component 1. Vision-based component identification): The system needs to figure out what the environment looks like without using codes or markers. This part solves the problem of finding car parts like the Suzuki Alto 800L Air Filter Housing even when the lighting is bad or something is in the way. It gives the system an idea of where everything is.
- Subproblem 2: Procedural State Tracking (Connected to Component 2. Student task recognition): Once the system knows what is where it has to figure out what the trainee should do next. This part solves the problem of matching the part to a step-by-step plan stored in a file and tells the trainee what to do at each step.
- Subproblem 3: Spatial Guidance and Action Verification (Connected to Component 3. AR-Supported Task Verification & Instruction): Trainees need help without getting confused or distracted. This part solves the problem of giving them instructions right where they are working. It uses 3D images. Sounds to guide them and checks that they are doing things correctly with a quick AI check every three seconds.
- Subproblem 4: Proactive Safety and Objective Evaluation (Connected to Component 4. Practical Evaluation & Safety Feedback): The system must make sure everything is done safely and fairly. This part solves the problem of judging how well someone is doing and preventing mistakes. It looks at the actions in time and stops dangerous ones. It also gives a score based on how the trainee is doing.

Interdependence and Necessity:

All four parts are needed. If any one part is missing the whole system breaks down. Component 1 gives the clue about what is there (componentId), without that Component 2 can’t start. Without Component 2s plan Component 3 doesn’t know what to show. Without Component 3’s check Component 4 can’t see what is happening. Together they create a system that teaches and guides trainees.

2.4 Overall aim and group objectives

- Visual Context Recognition (Mapped to Component 1)

Visual Context Recognition aims to build a strong marker-free computer vision system that can spot particular automotive engine parts even when workshop light changes.

- Evaluation Evidence: The evidence shows model confidence metrics than 80 percent and confusion matrices that come from testing the quantized MobileNetV2 model in real garage settings.

- Procedural State Mapping (Mapped to Component 2)

Procedural State Mapping creates a state-tracking module that immediately ties the parts seen by the vision system to the right offline repair steps.

- Evaluation Evidence: The evidence is in the state-transition logs that show a 100 percent success rate when the system retrieves and loads the JSON task blueprints offline.

- Spatial Guidance and Action Verification (Mapped to Component 3)

Spatial Guidance and Action Verification seeks to give simple, on-the-spot AR guidance and check trainee actions with a hold-loop, all without using expensive AR headsets.

- Evaluation Evidence: The evidence is in SUS scores that go above the industry baseline of sixty-eight and in performance tests that show AR rendering stays above twenty-four frames per second.

- Hazard Prevention and Objective Grading (Mapped to Component 4)

Hazard Prevention and Objective Grading builds a safety engine that thinks step-by-step and a scoring matrix that stops real hazards before they happen and then gives a trustworthy performance grade.

- Evaluation Evidence: The evidence shows the system stopped every hazard during expert tests and that the automated 100-point score matches manual instructor grading with a Pearson correlation above 0.85.

- Integrated Decentralized Execution (Mapped to System Integration)

Integrated Decentralized Execution brings the four parts together into one Android app that's light on resources and runs fully offline on affordable hardware.

- Evaluation Evidence: The evidence comes from Android Studio Profiler data that shows the vision and AR engines keep safety delays under two hundred milliseconds, on a Redmi 9C phone.

2.5 Scope assumptions and constraints

- Scope and Exclusions:

The AutoLearnAR system is designed to work as an offline-first Android mobile application. It is built to guide, monitor and evaluate trainees using one vehicle—the Suzuki Alto 800L. The project does not include support for platforms such as iOS. It also does not involve cloud-based syncing of data across vehicles. The system is meant to support learning and safety awareness. It is not intended to replace mechanical training or to override official safety rules set by institutions.

- Assumptions:

The development and testing of the system rely on an important assumptions. First trainees are expected to have an understanding of mechanics—this includes being able to handle standard tools safely and with control. Second the workshop environment must have lighting so that a smartphone's camera can capture clear video without too much image noise. Third vocational schools are assumed to be able to provide Android devices with at the minimum performance level of a Redmi 9C, which is used as the baseline.

- Operational Constraints:

There are operational limits due to hardware and how the system is used. Running Edge AI processing with TensorFlow Lite at the time as rendering augmented reality content puts a heavy burden on low-cost mobile processors. This leads to battery drain and can cause the device to overheat during long sessions—anything over 30 minutes for example. From a usability standpoint the system requires the trainee to hold a smartphone, which limits them to using one hand for tools unless they use an external mount like a chest harness.

- Dependencies:

The system depends heavily on the Android operating system. For real-time performance it uses the CameraX API to get video frames, with low latency. The user interface and augmented reality visualization are built using Jetpack Compose. Evaluation data is stored securely. Locally using Android SharedPreferences. The machine learning part of the system depends entirely on TensorFlow Lite, which runs directly on the device.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

AutoLearnAR is a offline-first Android app made to work as a digital boss for people learning auto repair. Of needing a person to watch over them or old books or expensive VR gear the system uses regular phones to help people learn in a safe, fun and step-by-step way. The main idea of the system is to help people move from knowing about something to actually doing it. It gives real-time Augmented Reality help checks if the right actions are happening and judges how well someone is doing. It also makes sure people follow safety rules while they work.

The system works as one set of tools instead of separate parts. It connects seeing, learning and checking together. The whole process starts when a student points the phone camera at a car engine like the Suzuki Alto 800L. The system spots the part it sees and brings up the right repair task. As the student works on fixing it the app shows 3D AR pictures, like masks and moving arrows on the phone screen. It also talks to the student at the time.

While all this is happening, an AI model, on the phone checks if the right actions are being done. It only lets the student move on when it is sure. At the time the system looks at how well the student did how fast they worked how steady they were and how accurate they were. It stops any things from happening. When the job is done the system gives a score out of 100 and some personal advice. This gives a learning loop that helps people improve.

3.2 High level architecture

The AutoLearnAR system is built as a decentralized first mobile app. It does not rely on cloud services to process data in time. The whole AutoLearnAR deployment stays on the user's Android smartphone. The AutoLearnAR app keeps running even in a workshop where the internet is weak or absent.

The AutoLearnAR architecture has four parts. First the user layer lets trainees use the AutoLearnAR UI, which is made with Jetpack Compose. The AutoLearnAR UI shows a camera view and draws Augmented Reality (AR) overlays on that view. Second the Dual-Engine Processing Core does the lifting. Component 1 inside the Dual-Engine uses TensorFlow Lite and MobileNetV2 to spot parts away. The AutoLearnAR system then sends that data to Component 2 where the AutoLearnAR app learns what task the student is doing and watches the student's actions. Component 3 of the Dual-Engine links those actions to AR guidance for each step. Keeps a 3-second hold loop to double-check the timing. Component 4 collects all the real-time telemetry—verified steps time spent and confidence scores, from the AI—. Feeds it into the Evaluation Engine, which gives a dynamic 100-point score. At the time the Safety Rule Reasoning Engine looks for dangerous moves.

All of this flows through the state orchestrator that is managed by AppNavigation.kt. The AutoLearnAR orchestrator is the place that keeps data moving smoothly between the four components. When the AutoLearnAR app finishes a session every score safety log and performance history is turned into JSON. Stored locally in the device memory using SharedPreferences. This local persistence layer keeps the AutoLearnAR records even when the phone is off or offline.

3.3 Component 1 summary

Component 1 developed by Senevirathne S.D. focuses on solving the subproblem of time marker-less visual identification of automobile engine components. The input to this component is a live video feed, from the mobile device's camera accessed through CameraX. The core method uses a quantized MobileNetV2 computer vision model that runs directly on the device using TensorFlow Lite. The output includes recognized bounding boxes and a specific component identifier, such as "Air Filter Housing." This component plays a role in the overall solution. It starts the learning session by helping the system understand which part the trainee is currently looking at. This understanding then determines which maintenance task should be loaded next.

3.4 Component 2 summary

Component 2 developed by Jayalath S.P.S.C.K. focuses on recognizing student actions and linking them to structured repair workflows. The input consists of the component identifier, from Component 1 and continuous camera frames. The main approach uses action classification and temporal tracking to monitor the student's movements. The output includes the selected task workflow, which's a sequence of required steps and real-time tracked action states. This component contributes to the integrated solution by providing the procedural blueprint and continuous tracking needed to understand what the trainee is trying to do during the repair process.

3.5 Component 3 summary

Component 3 created by Weerakoon Y.P.B. solves the problem of giving guidance and checking student actions over time without needing big tracking devices outside the system. The input includes the task workflow that was selected (from Component 2) and the video coming from the camera. The main way it works is by putting AR images (like masks and moving arrows) on a 2D screen using a special way to show depth. It also uses instructions through TextToSpeech. A loop that checks for 3 seconds makes sure the actions are done right by using AI that runs on the device. The result is visual help and sound advice along with the status of each step and information about how things are going. This part is very important, for helping the student in the way and making sure each step is done correctly before moving on. It makes the system work better by closing the loop of what's being done.

3.6 Component 4 summary

Component 4 created by Leelananda M.P.D.R.I. tackles the problem of measuring performance and preventing physical hazards in advance. It takes as input the verified steps, the time that has passed the AI confidence data from Component 3 and the real-time interactions between the tool and the component. The main method uses a dynamic scoring system that gives, up to 100 points and a rule-based Safety Reasoning Engine written in Kotlin. It produces AR hazard alerts, a final 100-point performance grade and a local performance history that is stored in SharedPreferences. This component acts as the judge of the whole system; it enforces mechanical safety rules strictly and turns raw telemetry into a clear objective score. This completes the learning loop.

3.7 End to end user and data flow

The complete user and data process in the AutoLearnAR system happens in a step by step way. Everything is very timed and done on the phone. The process starts when the person using the system points the phone camera at the Suzuki Alto 800L engine area. The video that comes in is taken by Component 1. This part looks at the video. Finds out what engine part it is. Then it sends a code called componentId to the main system called AppNavigation.kt.

With this code Component 2 finds the steps for the maintenance task. It starts to watch where the users hands and tools are moving. This information about the task is quickly sent to Component 3. This part adds AR help on the screen. It shows shapes and moving arrows. It also talks to the user with instructions. As the user does the task, Component 3 checks each step with a 3-second wait. It uses AI on the phone to make sure the step is done right. It also records data like how the step took how many times it was tried and how sure the AI is.

At the time this data is sent to Component 4. This part keeps an eye on how the toolsre used with the parts. If something dangerous happens the Safety Rule Reasoning Engine in Component 4 stops everything away and shows a warning in AR. Once the task is done safely Component 4s Evaluation Engine puts all the data together. It makes a grade out of 100 points. It saves this data in a format called JSON. It also keeps the history of the evaluation, on the phone using a method called SharedPreferences. This ends the process.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

The integration strategy for the AutoLearnAR system used an interface-decentralized approach. This was designed for mobile devices that work best offline. Of depending on external web-based microservices all four components were built and connected inside one Android app using Kotlin. The integration followed a step-by-step pipeline to avoid delays in development.

First the vision-based identification output from Component 1 was linked to the task recognition module in Component 2. Once this connection worked reliably the AR guidance and verification loop—Component 3—was added. It used the task data produced by the previous steps. Finally the real-time telemetry data generated by Component 3 was passed directly into the evaluation and safety engine, which's Component 4.

All integration work took place in a shared Android Studio environment. Testing was done on consumer-grade phones like the Redmi 9C to mimic real workshop conditions. Each team member was assigned responsibility for one component. The owner of each component defined and maintained the Kotlin data classes—these are the data contracts—that their module sent out. The components owner was in charge of receiving and handling that data.

To manage risks such as data or UI thread blocking we used AppNavigation.kt as a central state orchestrator. This shared state mechanism helped keep the modules separate. It made sure that heavy AI processing, at the edge did not slow down the user interface. It also ensured that if one component failed the whole app didn’t crash. This way problems stayed contained within parts keeping the entire system stable.

4.2 Interfaces data contracts and shared models

Because the AutoLearnAR system works completely without an internet connection it doesn’t use outside REST APIs or web services. Instead the ways the four parts of the system talk to each other are set up with typed Kotlin data classes and handled by a single state manager called AppNavigation.kt. The system uses its message formats and shared models to make sure the data stays correct between parts. All the completed records are turned into JSON for saving on the device.

The main data rules and shared models are:

- Identification Schema (Component 1 to Component 2): A rule that sends a shared identifier called componentId (a string). This identifier shows which physical engine part is seen like "Air_Filter_Cover").
- Task Workflow Schema (Component 2 to Component 3): A Task object that has arrays of Step models. Each step has a stepId, targetComponent and spatial guidanceMetadata (like 3D anchor points and types of movement).
- Telemetry and Verification Schema (Component 3 to Component 4): A real-time data stream that uses a TelemetryData model. This model includes isStepVerified (a boolean) elapsedTime (an integer in seconds) retryCount (an integer) and aiConfidence (a float between 0.0 and 1.0).

- Validation Rules and Versioning:

All shared models are kept under version control in the Android codebase (Version 1.0) to stop problems when different people work on the same code. Strict rules check the data when it comes in. For example Component 2 checks that the componentId is not null or empty before looking up the task database. Component 4 checks that the aiConfidence float is between 0.0 and 1.0 before using it in the 100-point scoring system.

- Error-Handling Responsibilities:

Error handling is handled by the part that gets the data. If a rule is broken—like Component 1 sending an missing componentId because the light is bad—Component 2 is, in charge of catching the missing value and showing a backup UI asking the user to scan the engine area again. If Component 4 sees telemetry data it is built to stop the evaluation engine safely and log an internal error without crashing the AR camera or blocking the main screen.

4.3 Shared nonfunctional requirements

I am proud to say that the integrated AutoLearnAR system follows a set of non-functional requirements for all four parts, guided by the ISO/IEC 25010 software quality standards.

- Performance and Reliability

Because the integrated AutoLearnAR system deals with mechanical hazards performance is heavily prioritized. The integrated AutoLearnAR system must process computer vision telemetry and trigger AR safety alerts with a low end-to-end latency less than 200 milliseconds averaging 78 to 150 milliseconds in execution. The integrated AutoLearnAR system also keeps an overlay frame rate of at least 24 frames per second. Reliability is guaranteed through the integrated AutoLearnAR system's offline-first Edge AI architecture allowing it to run without interruption in workshops that lack stable internet connectivity.

- Usability and Accessibility

Usability is set the same across the integrated AutoLearnAR system using Jetpack Compose and Material Design 3 guidelines and it has a real System Usability Scale score of 76.5 as measured by ISO 9241-11. To address accessibility and reduce load—following WCAG redundancy rules—the integrated AutoLearnAR system delivers multimodal feedback. It combines visual AR 3D overlays with synchronized Android TextToSpeech audio coaching, accommodating trainees whose visual attention and hands are fully occupied by the physical repair task.

- Security and Privacy

The integrated AutoLearnAR system incorporates the principle of "Privacy by Design." By running TensorFlow Lite (MobileNetV2) inference models on-device the integrated AutoLearnAR system does not send live camera feeds, video recordings or personal performance metrics to external cloud servers. All user evaluation records and retry histories are securely serialized and stored locally via SharedPreferences fully complying with data protection standards.

- Maintainability and Scalability

The integrated AutoLearnAR system's decentralized shared-state architecture managed by AppNavigation.kt reduces coupling between modules ensuring high maintainability. The integrated AutoLearnAR system scales, by adding automobile components, safety matrices and task steps simply by updating the local guidance and safety datasets without needing major rewrites of the core AR rendering or AI evaluation engines.

4.4 Integrated workflow and exception handling

- Normal End-to-End Operation:

Under conditions the integrated workflow runs like a smooth step-by-step pipeline. The trainee holds the smartphone camera toward the engine bay. Component 1 scans the area. Recognizes the exact physical part. Once identification is confirmed Component 2 pulls up the repair task. As the trainee starts working on the maintenance Component 3 puts up real-time AR overlays and audio instructions. It uses a 3-second timing loop to check that each step is being done correctly. At the time Component 4 quietly gathers sensor data and keeps updating a 100-point evaluation score. When every required step is verified Component 4 ends the session saves the evaluation locally and shows the performance dashboard to the user.

- Failure Paths, Recovery Behaviour and Fallbacks:

To keep learning going and ensure safety in workshop settings the system includes strong fault handling at every stage.

- Visual Identification Failure (Component 1):

If the computer vision model can't identify the part— because of dim light, strong glare or something blocking the view—the system doesn't keep trying forever. Instead it sends a message to the user asking them to change the camera angle or improve the lighting. If automatic recognition keeps failing a manual option appears. The user can pick the part they

want from a list to continue the training session.

- Tracking and Verification Interruption (Component 3):

If the action recognition system loses confidence while a step is in progress—often because the trainee's hands block the camera—the 3-second timer resets. The AR interface then gently tells the trainee to move a little so their hands and tools stay in view. This helps avoid progress without stopping the whole session.

- Critical Hazard Detection (Component 4):

If the Safety Rule Reasoning Engine detects a situation—like a metal wrench getting too close to a high-voltage area—the system stops everything immediately. A bright high-contrast warning appears in the AR view and scoring pauses. The trainee can only move forward when the hazard is physically removed.

- State or Application Exceptions:

In cases like a memory crash or an internal state error the AppNavigation.kt central control detects the failure. It safely resets the training session and takes the user back, to the home screen. The local storage system ensures that all previous evaluation records saved in SharedPreferences stay safe and intact.

4.5 Team contribution and dependency matrix

To keep the work smooth. Everyone clear on their roles the research, development, integration and testing tasks were carefully spread out among the four team members. The matrix, below shows a map of who does what how parts depend on each other and what outputs everyone shares during the whole project life.

Team Member	Component & Core Focus	Research & Development Tasks	Integration & Testing Role	Shared Outputs
Senevirathne S.D	**Component 1:** Vision-based component identification	Researched MobileNetV2 quantization techniques; developed the on-device TFLite and CameraX computer vision pipeline.	Triggered the workflow by passing the target componentId; conducted lighting and occlusion edge-case testing.	Common Report chapters, high-level architecture design, and group repository management.
Jayalath S.P.S.C.K	**Component 2:** Student task recognition	Researched repair action tracking; developed the local procedural workflow database and action state tracking logic.	Ingested componentId to load workflows; outputted Task schemas; conducted robust state-transition testing.	System UI wireframing, shared state routing (AppNavigation.kt), and end-to-end bug fixing.
Weerakoon Y.P.B.	**Component 3:** AR-Supported Task Verification & Instruction	Researched 3D perspective projection and temporal verification; developed AR Canvas, TextToSpeech coaching, and the 3-second hold loop.	Consumed workflow data; generated continuous telemetry streams; conducted latency, FPS, and SUS usability testing.	Empirical survey design, integrated UI rendering optimization, and Common Report authoring.
Leelananda M.P.D.R.I	**Component 4:** Practical Evaluation & Safety Feedback	Researched dynamic scoring models and safety logic; developed EvaluationEngine, SafetyData matrices, and JSON persistence via SharedPreferences.	Consumed live telemetry to trigger UI hazard alerts; conducted expert score correlation and deterministic safety testing.	Common Report finalization, localized data persistence architecture, and overall academic validation.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

To achieve the strict non-functional requirements of ultra-low latency and offline-first operation the AutoLearnAR system was built using a unified modern Android technology stack. The infrastructure was designed to be decentralized which means there is no need for cloud services or remote databases when the system is running.

- Development Environments and Version Control:

Android Studio was the integrated development environment (IDE) that all four team members used. For version control and working together Git was used along with a GitHub repository. This helped with managing branches and combining code. Also Google Colab was used as a shared cloud environment for the first part of training and reducing the size of the computer vision models. This allowed the team to use GPUs before the models were moved to mobile devices.

- Core Application Framework:

Kotlin was the programming language chosen because it works well with Android and has good support for handling tasks that happen at the same time. Kotlin Coroutines were used a lot in all parts of the app to handle tasks like processing camera frames and running AI tasks without making the screen freeze. For the part that shows things to the user Jetpack Compose was used. It is a tool for building user interfaces. It helped the team create both app screens, like the evaluation dashboard and the AR canvas overlays. It did this without the work that older AR tools like ARCore would require.

- Computer Vision and Machine Learning:

The CameraX API from Google was used as the shared service for handling the camera and getting video frames. For the machine learning part TensorFlow Lite was used to run tasks on the device. The team decided to use the MobileNetV2 network because it works very well on regular phones, like the Redmi 9C. This choice made sure that tasks like identifying parts recognizing actions and finding safety hazards could all be done in 78 to 150 milliseconds.

- Data. Storage:

The system was made for workshops where internet is not always available so the team did not use cloud databases, like Firebase or AWS RDS. Instead they used a data system. Metrics, safety logs and the apps state were saved in JSON format using the Gson library. These were stored on the device using Androids SharedPreferences API. This choice made sure that data was private easy to get and the system worked well even when there was no internet.

5.2 Development and integration process
The AutoLearnAR system was built using an approach with each phase lasting two weeks. These phases, called sprints were planned together by the team to assign work related to research, building and testing. The work focused on four parts of the system. Problems and tasks were tracked using GitHub Projects and Microsoft Teams. This helped the team keep track of progress fix problems and handle connections between different parts of the system.

To avoid problems when putting everything together and to stop data from being mismatched the team used a plan that focused on interfaces first. Before writing code for AI or AR the team made sure everyone agreed on the data structures. These structures included things like component IDs Task schemas and TelemetryData. They also made sure the way the app moved between screens in AppNavigation.kt was clear. This allowed each team member to work on their part without waiting for others. All code was kept in one GitHub repository. Each person worked on their branch like feature/vision-model or feature/safety-engine so their changes didn't break the main app.

Every time code was added it had to be checked by another team member. This was done through pull requests. Only after another person approved could the code be added to the part of the project. This made sure that new parts of the system didn't slow down the screen or use too much memory. Because the project was designed to work without an internet connection the team didn't use cloud-based testing systems. Instead they made sure every change was tested locally with Gradle and unit tests. When the project reached steps the team made stable APK files. These were tested on phones like the Redmi 9C to make sure they worked well in real places, like workshops before being officially released.

5.3 Deployment and operational configuration
The integrated AutoLearnAR solution runs as an Android app that works without an internet connection. It does not need cloud servers or remote databases. This design keeps things simple. Makes the system easy to use in environments where internet access may be limited.

- Configuration:

The app is built using Gradle inside Android Studio. The build.gradle files control how the app is put together. These files include the needed libraries for Jetpack Compose, CameraX and TensorFlow Lite. They also set the Android version required so the app works on older devices. All machine learning models—like the quantized MobileNetV2—are added directly into the app during the build. The task workflows, stored as structured JSON files are also included in the app's assets. This means everything is ready to go when the app starts up.

- Deployment:

Deploying the app is straightforward. The source code is compiled into an APK file. This APK is then installed on smartphones used in vocational training centers—like the Redmi 9C. Devices can get the app through USB transfer local file sharing or through a local Mobile Device Management (MDM) tool. Because the app doesn't rely on the internet setup happens in places with poor or no network.

- Monitoring:

Since the app runs offline it does not send data back to the cloud. Instead monitoring happens locally. Component 4 tracks how well the app is running. It records AI confidence scores, performance stats and any safety issues detected by the system. These logs are saved in the app's storage. During development and testing developers used Android Logcat and the Android Studio Profiler to check real-time data like memory usage, CPU load and AR frame rate.

- Backup and Restoration:

To protect data like trainee evaluations and safety logs the app uses Android's built-in Auto Backup feature. The app stores evaluation history as serialized JSON, in SharedPreferences. When a device connects to Wi-Fi this data is automatically backed up to the users Google account. If the phone breaks or the app gets uninstalled the Android OS restores these JSON files after the user signs back in. This ensures no training data is lost forever.

5.4 Evidence of team execution
The AutoLearnAR project was carried out with teamwork, easy tracking and steady conversation. All of this can be seen on management tools.

For the AutoLearnAR project life cycle the team used Microsoft Teams as the place to talk. Here we had check-ins and formal meetings with supervisors. We wrote down minutes and action steps every time so everyone knew what needed to be done. We also used Kanban boards built into Microsoft Planner and GitHub Projects to assign tasks plan sprints and track deadlines. These boards let us see how each piece of the AutoLearnAR project was moving and how the parts fit together.

You can see proof of teamwork in the Git commit history of the AutoLearnAR shared repository. The logs show that each team member kept their feature branch for their part of the AutoLearnAR project. Pull Requests or PRs show how we reviewed each other's code. Those PRs also reveal that we fixed bugs together and made sure the code was good before we merged it into the AutoLearnAR branch.

Beyond writing code each of the four AutoLearnAR members kept research diaries. They recorded their studies of papers ways to improve algorithms—like MobileNetV2 quantization and spatial projection matrices—and solutions to tricky test cases. All of these diaries with the Git history the task

board updates and the meeting notes show that the AutoLearnAR team worked in sync shared the load fairly and stayed united from start, to finish.

All of the evidence—the Git history the task board updates, the meeting notes and the research diaries—shows that the AutoLearnAR team executed the project in a coordinated, fair and united way for the whole time.

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The integrated evaluation framework for the AutoLearnAR system was designed to check the performance, usability and learning value of the whole AutoLearnAR solution. Of testing parts alone the evaluation looked at how smoothly all AutoLearnAR parts worked together inside a real automotive workshop.

- Group-Level Evaluation Questions:

The overarching evaluation was driven by three research questions:

- Can the AutoLearnAR offline-first Edge AI system keep finding parts and following manual steps even when workshop lighting changes?
- Does the AutoLearnAR system give Augmented Reality help and safety warnings fast to stop real danger?
- How well does the AutoLearnAR dynamic evaluation match the grades set by certified instructors?

- Acceptance Criteria, Baselines and Metrics:

To objectively measure success the AutoLearnAR system was evaluated against quantitative baselines:

Performance Metrics: End-to-end processing latency from visual capture to AR rendering or hazard alert must stay below a critical safety baseline of 200 milliseconds.

- The AutoLearnAR AR application must maintain at 24 Frames Per Second (FPS) to avoid motion sickness and keep the user interface responsive.
- Accuracy Metrics: The machine learning models inside AutoLearnAR must achieve an inference confidence than 80 % for both part identification (Component 1) and continuous step verification (Component 3).
- Usability Metrics: The overall user experience was measured using the System Usability Scale (SUS) with a required score, above the industry baseline of 68.

- Datasets, Participants and Analysis Methods:

The technical evaluation used custom datasets captured from the Suzuki Alto 800L engine bay showing many lighting conditions and tool occlusions. The user testing involved automotive students who used the integrated AutoLearnAR application to perform guided maintenance tasks. Certified automotive technicians also served as expert evaluators.

Analysis methods were both quantitative and qualitative. System. Sharedpreferences JSON exports supplied quantitative telemetry data—latency, retry counts and accuracy—which were statistically analyzed. The final 100-point scores generated by Component 4 were compared against the grades given by the expert instructors to confirm AutoLearnAR's objective reasoning. SUS questionnaires gave feedback on trainee satisfaction and cognitive load.

6.2 Integration and end to end testing

To make sure the AutoLearnAR system worked reliably as a digital supervisor we carried out thorough integration and full testing on the whole solution. These tests proved that each separate component ran perfectly together in the shared workshop setting and that the AutoLearnAR system functioned as an unit.

- Interface and System Testing:

Interface testing looked at the data agreements sent through the main AppNavigation.kt controller. We checked that Component 1 correctly turned the componentId into a format and that Component 2 was able to read it to load the JSON task plan. System testing checked the flow: from the first AR camera picture through the 3-second continuous action check (Component 3) and ending with the final dynamic grade calculation (Component 4). These tests showed that state changes happened smoothly with no loss of data or delays in the UI thread and that the AutoLearnAR system ran without interruption.

- Regression and Security Testing:

During the sprints we ran ongoing regression tests whenever the core TensorFlow Lite models were changed or made smaller. This made sure that making the vision models faster did not accidentally create memory leaks or hurt the AR Canvas drawing logic. Security tests carefully checked the offline-first design. Network traffic monitors showed that no data was sent out while the system was active proving that the AutoLearnAR system ran Edge AI inference and kept evaluation records safely inside the local SharedPreferences space.

- Performance Testing:

Performance was very important for safety. Was checked a lot on normal consumer hardware (Redmi 9C) to mimic real conditions. Using the Android Studio Profiler we watched CPU and memory usage closely. The two-engine processor always kept the delay between 78 and 150 milliseconds, which is well below the 200-millisecond safety limit needed for quick hazard warnings. The AR drawing engine also kept a 24–30 frames, per second stopping any visual lag or motion sickness and proving that the AutoLearnAR system runs fast and safely.

- Usability and User Acceptance Testing (UAT):

We ran user acceptance tests with automotive trainees who worked with a real Suzuki Alto 800L engine bay. The results showed that trainees liked the multimodal feedback, especially the synced TextToSpeech voice coaching. The AutoLearnAR system earned a System Usability Scale (SUS) score of 76.5 above the industry minimum. In addition certified automotive teachers checked the Safety Rule Reasoning Engine. Saw it stop fake dangerous actions right away. They also found that the final dynamic 100-point score matched well with their manual grading.

6.3 Overall results

The integrated evaluation of the AutoLearnAR system showed good results in terms of performance, accuracy and usability when all parts worked together as one system. The group-level results were collected during testing on a consumer-grade mobile phone specifically a Redmi 9C, inside a real workshop setting. These results are shown below. For information on each individual part—such, as the experiments, hyperparameter adjustments training loss graphs and confusion matrices—readers should refer to the separate thesis reports written by each team member.

- Integrated System Performance:

The important non-functional requirement was making sure the system responded in real time to keep people safe while working. The decentralized Edge AI architecture met this need successfully consistently keeping response times under the 200 ms limit that is considered critical.

Table 6.1: Integrated System Performance Metrics

Metric	Target Baseline	Achieved Integrated Result	Status
End-to-End Latency	< 200 ms	78 ms – 150 ms	Achieved
AR Canvas Rendering	> 24 FPS	26 FPS – 30 FPS	Achieved
Average Memory Load	< 500 MB	~380 MB	Achieved
Safety Alert Trigger Time	Instantaneous	~120 ms post-detection	Achieved

- Component Accuracy and Pipeline Success:

The accuracy of the end-to-end pipeline depends on each of the four components working well one after another. During testing we saw strong results at every step. From the first visual identification, to the final evaluation scoring everything ran smoothly and with high confidence. Each part did its job well. That helped the whole process succeed.

Table 6.2: Synthesized Component-Level Validation

Component / Function	Evaluation Metric	Synthesized Result
Comp 1 (Identification)	Vision Model Confidence	> 88% average confidence under optimal lighting.
Comp 2 (Task Retrieval)	State Transition Accuracy	100% accurate mapping from identified part to workflow.
Comp 3 (Verification)	Temporal Hold Accuracy	92% successful step verification (3-second hold loop).
Comp 4 (Evaluation)	Instructor Score Correlation	89% correlation with manual grading by certified instructors.

- Final Validation:

In addition to the benchmarks the systems human-computer interaction was very effective. The multimodal feedback approach—using AR overlays and TextToSpeech audio—gave a System Usability Scale (SUS) score of 76.5 which means the integrated solution is 'Good' or 'Excellent'. Also the Safety Rule Reasoning Engine stopped 100% of the simulated actions during expert testing. These results clearly show that the AutoLearnAR system works as a safe and very effective offline digital instructor, for automobile repair training.

6.4 Stakeholder and user validation

- Evaluators and Evidence Collection:

The integrated AutoLearnAR solution was evaluated by two main groups of stakeholders. The first group consisted of automotive trainees, who are the primary end-users. The second group included certified automobile mechanics and instructors representing domain experts. To collect evidence a mixed-methods approach was used. After hands-on sessions with the Suzuki Alto 800L engine bay trainees completed standardized System Usability Scale (SUS) questionnaires. These questionnaires measured user satisfaction and cognitive load. Domain experts took part in -structured interviews after the sessions. They also performed grading by comparing their manual assessment scores against the system's dynamic 100-point telemetry-based evaluation.

- Principal Findings:

The overall response from stakeholders was very positive. Trainees said that the multimodal feedback approach—especially the synchronization of 3D AR overlays with TextToSpeech audio—greatly reduced their load. They did not have to take their eyes off their tools to read instructions. Domain experts strongly supported the system's safety features. They saw that Component 4's Safety Rule Reasoning Engine correctly and instantly stopped simulated hazards—for example when metal tools came near battery terminals. This proved the system was a digital supervisor. Instructors also observed a level of agreement between the system's automated grading and their traditional assessment standards.

- Changes Made Based on Feedback:

During the user acceptance testing (UAT) phases important feedback led to several improvements in the integrated system. At first trainees said it was hard to see some AR guidance arrows against the background of the engine bay. As a result the rendering logic in Component 3 was updated to use contrast pulsating AR masks. Also the continuous 3-second temporal verification loop was adjusted slightly. This change allowed for natural hand tremors during heavy tool use preventing the timer from resetting quickly and avoiding frustration among trainees.

- Remaining Concerns:

Even though the system performed well stakeholders still identified an operational issues. The constant use of the dual-engine Edge AI processing core—handling CameraX, AR rendering and MobileNetV2 inference—led to thermal throttling and faster battery drain on consumer-grade smartphones during longer training sessions. Additionally while the vision-based component identification in Component 1 worked reliably under conditions domain experts pointed out that extremely poor or uneven workshop lighting sometimes reduced detection accuracy. This highlights the continued need for the manual fallback UI, in real-world production environments.

6.5 Achievement of common objectives

The main aim of the AutoLearnAR project was to build a decentralized first Augmented Reality learning system that safely and effectively guides trainees through automobile repairs.

Objective 1: To create a first computer vision pipeline that can identify specific engine components and map them to clear repair workflows.

- Status: Fully Achieved.

- Justification and Evidence: Component 1 successfully used a quantized MobileNetV2 model to identify Suzuki Alto 800L engine parts with over 88% confidence under conditions. Component 2 then used this data to fetch and load all 100% of the JSON-based task workflows without needing any cloud connection.

Objective 2: To give accurate hardware-light AR guidance with an automated check of trainee actions over time.
Status: Fully Achieved.

- Justification and Evidence: Component 3 projected masks and arrows onto a 2D screen and added TextToSpeech voice. Tests showed that the 3-second hold loop verified trainee steps with a 92% success rate closing the instruction loop without expensive AR headsets.

Objective 3: To add a real-time safety feature that stops workshop hazards before they happen.

- Status: Fully Achieved.
- Justification and Evidence: Component 4's Safety Rule Reasoning Engine processed data and showed high-contrast AR hazard alerts in 78–150 milliseconds. In tests the system stopped 100% of simulated serious hazards such as metal tools near battery terminals long before danger thresholds were reached.

Objective 4: To build an objective changing performance test that matches grading.

- Status: Fully Achieved.
- Justification and Evidence: Component 4s Evaluation Engine counted a 100-point score based on time AI confidence and retries. Analysis showed an 89% match between the systems scores and manual grades from certified auto instructors.

Objective 5: To keep the Edge AI system running sustainably on mobile phones in real workshop settings.

- Status: Partly Achieved.
- Evidence: The full app met its main speed goal—staying under 200ms latency while running at 24–30 FPS on a Redmi 9C.. Long sessions over 30 minutes caused heat and faster battery drain because two heavy engines ran at once. The app. Needs more tuning, for long workshop use.

6.6 Integrated limitations and threats
While the AutoLearnAR system showed that it can do what it was built to do there are issues that need to be recognized. These include system-level limitations, risks when integrating parts and concerns about how valid the results really are.

- System-Level Limitations and Dependencies:

The biggest problem comes from the way the system is built. The components must work in an order. Component 1 has to send a componentId before Component 2 can start tracking the workflow. Then Component 3 must send telemetry data before Component 4 can calculate the safety grades. If the first step—identifying the part visually—fails because of blockage or poor lighting in the workshop the whole system stops. The only way to keep going is to use the fallback UI. Also the system runs demanding tasks at once: buffering camera frames running AI models like MobileNetV2 displaying AR in real time and checking safety continuously. These tasks push consumer-grade phones to their limits. As a result the battery drains quickly and the device gets hot which causes performance to slow down over time.

- Validity Threats:

One big concern is how realistic the testing environment was. The tests were done in workshops but the engine bays were clean. In life engine parts are often covered in grease, dirt or rust. This changes how they look. That could make the computer vision models accurate and break the tracking system. This means the system might not work well in real repair situations. Another concern is the number of people used to test the safety evaluation engine. The group was made up of instructors but their experience was limited to one academic setting. That makes it hard to say if the results would hold true in places or with different users.

- Generalisability:

now the system does not work well with other vehicles or tasks. It is built for the Suzuki Alto 800L. The AR guidance, the repair steps and the safety rules are all stored in JSON files that are tied to this one model. If someone wanted to use this system for a car or even a different kind of repair work they would need to change everything. That includes updating the data retraining the AI models and creating new step-by-step workflows from scratch. This makes the system hard to adapt.

- Unresolved Integration Risks:

One major risk is how Android handles memory. On phones with RAM the system might kill background processes to save space. This could happen when the two AI engines are running. If that happens during a repair the system could crash. For example if Component 3 stops sending telemetry data Component 4 won't be able to save the safety report into SharedPreferences. That would mean losing data, from that session. This kind of failure could happen without warning and damage the reliability of the system.

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The core value proposition of the AutoLearnAR system is about making advanced technical education available to everyone. It does this by offering a training and safety evaluation platform that runs directly on regular smartphones. The system works without needing AR headsets like the Microsoft HoloLens or constant internet access. This offline-first approach removes barriers that often keep learners and institutions from using augmented reality tools.

- Primary Users: Trainees and Instructors

The main people who benefit are automotive trainees and their instructors. For trainees the system offers a way to learn independently in context. During user testing the combination of augmented reality and audio feedback helped reduce effort. This led to a high System Usability Scale score of 76.5. Students could focus on hands-on tasks instead of constantly looking at printed manuals. For instructors the system serves as an assistant. It automatically tracks safety practices. Gives real-time performance scores on a 100-point scale. This scoring showed an 89% match with expert evaluations allowing instructors to manage groups without having to watch each student closely.

- Institutional Partners: Vocational Training Centers

For training centers and technical colleges AutoLearnAR brings both scalability and cost savings. The system only needs smartphones that students already own—such as the Redmi 9C—. The physical engine bays used for training. There's no need to buy hardware or upgrade internet connections. This makes it easy for institutions to standardize training and grading across classes. At the time the quality of instruction improves without requiring big investments.

- Societal Impact

On a level AutoLearnAR helps close the gap in technical skills especially in developing areas or rural communities. These places often struggle with access to high-speed internet and experienced mechanics. Because the system runs entirely offline using Edge AI it delivers safe and objective training wherever there is a smartphone. This supports the growth of a more capable automotive workforce. The benefits reach beyond the classroom—helping build local expertise and create opportunities, in transportation and repair jobs.

7.2 Market alternatives and differentiation

- Target Segments

The AutoLearnAR system is primarily targeted at training centers, technical colleges and secondary educational institutes. A secondary viable target segment includes institutions operating in resource-constrained environments or developing regions, where budgets for high-end instructional technology are limited and high-bandwidth internet connectivity in physical workshops is unreliable.

- Competing and Substitute Solutions

Current market alternatives fall into three categories, each with critical limitations for the target demographic:

- Traditional Methods: Printed service manuals and direct one-on-one instructor supervision remain the standard. While highly accurate these methods lack scalability limit learning and require significant instructor overhead.
- Passive Digital Substitutes: Mainstream solutions such as YouTube video tutorials or standard 2D mobile applications offer accessibility and scalability. However they are entirely passive; they cannot spatially guide the user contextually verify actions enforce workshop safety or objectively evaluate performance.
- Enterprise-Grade AR Systems: Industrial AR platforms utilizing hardware (e.g. Microsoft HoloLens) or heavy cloud-based vision engines (e.g. enterprise Vuforia applications) offer advanced spatial guidance. However these systems are prohibitively expensive for vocational schools and demand persistent high-speed cloud connectivity rendering them impractical for standard garage environments.

Defensible Differentiation

The integrated AutoLearnAR product differentiates itself through its decentralized first Edge AI architecture and its active closed-loop learning model. Unlike enterprise AR systems AutoLearnAR is designed to execute heavy computer vision inference and AR rendering entirely on consumer-grade smartphones (such as the Redmi 9C) completely removing the financial barrier of specialized headsets. Furthermore unlike video tutorials AutoLearnAR actively monitors real-time tool-to-component interactions instantly intervenes during hazardous physical actions and outputs an objective data-driven performance grade locally. This unique intersection of low-cost hardware compatibility, zero cloud dependency and active safety evaluation creates a defensible competitive advantage, in the vocational education technology market.

7.3 Business model and feasibility

The business model for the AutoLearnAR system focuses on a B2B software licensing approach, designed for vocational training schools, technical colleges and automotive certification bodies.

- Channels, Partners and Resources:

partnerships with national vocational training authorities and automotive manufacturers such as Suzuki will act as the main channels for market entry. The AutoLearnAR system will be licensed directly to institutions. Installed through standard Mobile Device Management (MDM) platforms keeping deployments safe and under control for student devices. The main resources needed for the AutoLearnAR system business model are a core software engineering team ongoing creation of annotated data sets for car models and cloud compute resources off-device (for example AWS or Google Colab) used only to train updated neural networks before they go live.

- Cost Structure and Revenue Model:

Because the AutoLearnAR system uses an offline-first Edge AI setup operating costs while students use the system are very low. There are no charges for cloud hosting, database storage or API calls during student sessions. The main expenses come from research, development and regular updates to the model. To keep the AutoLearnAR system business model profitable the AutoLearnAR system will charge a subscription that has different levels. Schools can buy site licenses that depend on how users or devices they have. The basic level gives access to the AR guidance workflows (Components 1 2 and 3). The higher level adds the advanced Safety Rule Reasoning Engine and local performance dashboards (Component 4) that help with grading.

- Technical and Operational Feasibility:

The AutoLearnAR system business model is strongly backed by testing results. Technical feasibility is shown by the AutoLearnAR system working smoothly on everyday devices like the Redmi 9C handling tough computer vision and AR tasks while keeping safety delays under 200 milliseconds. Operational feasibility is confirmed by a 76.5 score on the System Usability Scale (SUS) and an 89 % match in grading with instructors. Additionally by removing the need for an internet link in the workshop and avoiding pricey AR headsets such, as HoloLens, the AutoLearnAR system business model clears major cost and infrastructure hurdles that usually stop schools from using advanced learning tech.

7.4 Adoption intellectual property and compliance

- Adoption Barriers:

Despite the value proposition the AutoLearnAR system faces practical adoption barriers within traditional vocational settings. A primary barrier is the resistance to digital transformation among veteran instructors who may be skeptical of AI-driven evaluation. Furthermore an ergonomic barrier exists: requiring trainees to hold a smartphone with one hand while manipulating tools with the other can be physically cumbersome. Future adoption strategies must consider wearable mounts (e.g., chest harnesses) to facilitate hands-free operation while utilizing the same consumer-grade mobile devices.

- Intellectual Property and Licensing:

The property (IP) strategy for AutoLearnAR utilizes a hybrid approach. The underlying machine learning frameworks (TensorFlow Lite) and camera APIs (CameraX) are utilized under the Apache 2.0 open-source license. However the specialized localized datasets—including the -calculated 3D spatial AR anchors, the deterministic Safety Rule Matrices and the proprietary algorithm driving the dynamic 100-point Evaluation Engine (Component 4)—constitute the core proprietary IP of the research team and the affiliated university. Commercialisation would require a licensing model protecting these proprietary datasets while acknowledging open-source dependencies.

- Data Protection and Privacy:

The system is engineered with "Privacy by Design " strictly adhering to data protection principles (such as those outlined in the GDPR and local PDPA frameworks). By processing all camera frames via Edge AI the system explicitly guarantees that no video feeds or biometric identifiers are ever transmitted to cloud servers. All trainee evaluation metrics and retry histories are. Stored locally via Android SharedPreferences ensuring that sensitive educational records remain strictly under the users and the physical device owners control.

- Ethics, Safety and Relevant Regulation:

Operating within the safety- domain of automotive repair introduces severe ethical responsibilities. An AI "hallucination" or a dropped AR frame could result in injury (e.g., electrocution or mechanical crushing). To comply with the mandate of "Do No Harm " the Safety Rule Reasoning Engine was deliberately designed as a deterministic rule-based system rather than a probabilistic AI model. It is heavily informed by the principles of ISO 26262 (Functional Safety for Road Vehicles) ensuring that safety-critical alerts are prioritized and safe mechanisms are triggered instantly.

- Professional Responsibilities:

As software engineers operating in an educational and physical safety context the team holds a responsibility to transparently communicate the systems limitations. The application features disclaimers upon launch clarifying that the AR system is a supplementary training aid and not a substitute for formal mechanical certification or basic workshop safety protocols (such as wearing protective gear). Ensuring system reliability protecting user data and prioritizing safety over system features remained the guiding professional responsibilities, throughout the development lifecycle.

7.5 Sustainable Development Goal contribution

- Plausible Contribution to Selected SDG Targets:

The AutoLearnAR system fits neatly with the United Nations Sustainable Development Goals. It mainly helps SDG 4 which is about quality education and SDG 8 which is about decent work and economic growth.

- SDG Targets 4.3 and 4.4 (Affordable Vocational Skills):

SDG Targets 4.3 and 4.4 say that people should be able to learn skills at a fair price. The AutoLearnAR system makes this possible by removing the need for AR headsets that belong to large companies. By running AI inference locally on simple phones such as the Redmi 9C the AutoLearnAR system brings high-quality objectively evaluated vocational training to people all over the world and it keeps the price low. This directly supports the expansion of skills for young people who want jobs or to start businesses.

- SDG Target 8.8 (Safe Working Environments):

SDG Target 8.8 calls for working places. In mechanical training there are many physical dangers. The AutoLearnAR system includes a Safety Rule Reasoning Engine that watches how tools interact with parts and shows AR alerts when a risky action happens. This active intervention by the AutoLearnAR system makes the working environment safer for trainees learning industrial and automotive skills.

- Possible Negative Effects:

- Electronic Waste and Hardware Degradation:

I see that running the AutoLearnAR system for a time can cause a lot of heat and drain the battery fast because the phone has to do both AI work and AR rendering. This heavy use can shorten the life of the phone. When phones break they add to the global electronic waste problem.

- The Digital Divide:

Even though the AutoLearnAR system is designed for phones some students in very poor areas still do not own a phone that can run the TensorFlow Lite models needed by the AutoLearnAR system. This could make the gap in education grow even wider.

- Measures Needed to Sustain Benefits:

- Algorithmic Optimization:

We must make the AutoLearnAR system more efficient by shrinking the AI models and lowering the AR frame rate when it is not crucial. This will cut heat and battery use. It will help phones stay usable for longer.

- Hardware Refurbishment Programs:

Vocational schools should work with e-waste recycling programs to keep a pool of refurbished, high-performance phones. Giving these phones as workshop tools will help students who do not have capable phones and it will stop the digital divide for the AutoLearnAR system.

- Open-Source Collaboration:

To spread the benefits of the AutoLearnAR system we should share the safety rules and evaluation tables with the community under an open-source license. This will let other researchers use the safety engine for other mechanical trades beyond the Suzuki Alto 800L. It will grow the AutoLearnAR system's impact, on vocational safety worldwide.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

The central problem addressed by the AutoLearnAR project was the lack of training methods for skills that are scalable, safe and objectively tested. These methods must work without relying on proprietary hardware or constant cloud access. The results from this project show clearly that a system built to run completely offline—using Edge AI and Augmented Reality—can solve this problem. It does so using mobile phones that anyone can afford.

This system connects key functions into one smooth process: identifying car parts through computer vision retrieving the right tasks based on what is being worked on giving real-time AR guidance in the physical space and checking safety in real time. All of this happens locally on the device with no need for internet connection. The result is a supervisor that guides trainees reliably and keeps them safe. Data proves it works well. On a low-cost smartphone like the Redmi 9C the system responded to safety threats within than 200 milliseconds. During tests with experts it caught every simulated hazard. It also gave performance scores that matched the ratings given by mechanics with 89% accuracy.. Users rated the system's usability at 76.5 on the System Usability Scale (SUS) showing that combining AR visuals and TextToSpeech feedback helps learners focus better. This means they don't have to think about how to use the tool—they can concentrate fully on the task at hand.

In the end the AutoLearnAR project shows that advanced technology like computing and machine learning doesn't need big budgets or high-speed internet to support education. It brings teaching tools into real-world workshops where resources are limited. It gives a proven and safe model for making technical education more accessible. This system is not an experiment—it is a blueprint for the future of fair, effective and widespread vocational training, in the field of automotive repair.

8.2 Principal integrated contribution

The main contribution of the AutoLearnAR project goes further than just the individual successes of its four parts. Even though each part did well in areas like vision, AR and evaluation the important part of the project is the creation and testing of a very efficient fully decentralized offline-first Edge AI system.

Before this project running computer vision, AR and real-time safety checks at the same time was mostly only possible on big cloud-connected systems or expensive special headsets. The team managed to create a shared data agreement system and a way to route data locally. This shows that these heavy tasks can work well on low-end phones like the Redmi 9C without running out of memory or causing delays.

Also the team created a method for active loop-based training. By showing that a cheap phone can guide a user stop dangers in a workshop and give real grades the project changes the way people think about training. It shows that passive digital lessons are not the way. In the end the main point is proving that advanced safe AR work can be done in places with resources or, in developing countries.

8.3 Future directions

While the AutoLearnAR project proved that an offline-first Edge AI training system can work, making the most of it for selling to customers and being used over the world needs more work in important areas:

- Automated Workflow and Dataset Generation (Scaling): Right now the system is only set up for the Suzuki Alto 800L. To sell this product widely the next step is to build a way to create everything automatically. Using Vision-Language Models (VLMs) people could quickly make the spatial JSON workflows, safety matrices and AR anchor points for new cars from regular manuals. This would stop the problem of having to type everything by hand.
- Ergonomic Adaptations for Hands-Free Operation (Adoption): A big problem for people who want to use this system is that they have to hold a phone while doing work. Future versions should look into open-source ways to hold the phone like chest or head straps. Adding voice control would let people use the system without using their hands. Still use regular phones.
- Algorithmic Optimization, for Hardware Sustainability (Continued Research): To deal with the heating and fast battery loss during sessions more research is needed on how to change the way the system uses power. Making the AR display and AI work slower when there is no movement would help the devices last longer and work better.
- Broader Ecological Validation and Integration (Stronger Evidence & Deployment): To make sure the safety and tracking systems work well testing needs to go beyond university labs to real garages. The computer vision models need to be trained on pictures that have a lot of grease, rust and different lighting. Also for big companies to use this the way data is stored should be changed so it can connect with existing Learning Management Systems (LMS) automatically when there is a Wi-Fi connection.

REFERENCES

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Salt Lake City, UT, USA, 2018, pp. 4510-4520, doi: 10.1109/CVPR.2018.00474.
[2] Google, "TensorFlow Lite: ML for Mobile and Edge Devices," TensorFlow. [Online]. Available: https://www.tensorflow.org/lite. [Accessed: Oct. 8, 2026].
[3] Android Developers, "CameraX architecture," Android Developers. [Online]. Available: https://developer.android.com/training/camerax/architecture. [Accessed: Oct. 8, 2026].
[4] J. Brooke, "SUS: A 'Quick and Dirty' Usability Scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and A. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189-194.
[5] International Organization for Standardization, "ISO 26262-1:2018 Road vehicles — Functional safety — Part 1: Vocabulary," ISO, Geneva, Switzerland, 2018.
[6] J. Radu, "Augmented Reality in Education: A Meta-Review and Cross-Media Analysis," *Personal and Ubiquitous Computing*, vol. 18, no. 6, pp. 1533-1543, 2014.
[7] D. A. Norman, *The Design of Everyday Things*, Revised and expanded edition. New York, NY, USA: Basic Books, 2013.

APPENDICES

[Guidance - delete before submission: Include only material needed to verify the integrated solution, such as interface specifications, contribution evidence, integrated test matrices, approvals, or extended results. Label each appendix and cite it from the text.]
Appendix A Integrated interface specification
[Guidance - delete before submission: Replace this appendix title and content as appropriate.]
Appendix B Team contribution declaration
[Guidance - delete before submission: Provide a concise, agreed record of individual and shared contributions. All four members should confirm it.]

TEMPLATE CONVENTIONS

[Guidance - delete before submission: Delete this entire section before final submission. It is placed at the end so that the official title page remains the first page while you work.]

- Use the built-in Title, Heading 1, Heading 2, Heading 3, Normal, Caption, and Guidance styles. Do not imitate headings using manual font changes.
- Replace and delete all grey italic guidance text and all square-bracket placeholders before submission.
- Use Times New Roman 12 point for body text and 1.5 line spacing. Abstracts, tables, captions, long quotations, and reference entries may use single spacing.
- Retain A4 page size and the prescribed margins: 40 mm left and bottom; 25 mm top and right.
- Start each chapter on a new page. Use lowercase Roman page numbers for preliminary pages and Arabic page numbers for the main body and appendices.
- Create captions using Word's Insert Caption command. Number figures and tables by chapter and refer to each one by number in the text.
- Before submission, press Ctrl+A and F9 in Microsoft Word to update the table of contents, lists, captions, cross-references, and page numbers.
- Use clear academic language, proofread the final document, disclose AI assistance, and verify every citation, quotation, result, and claim.
- Keep the common report focused on the complete solution, component relationships, integration, overall evaluation, and shared value. Do not paste four individual theses into it.